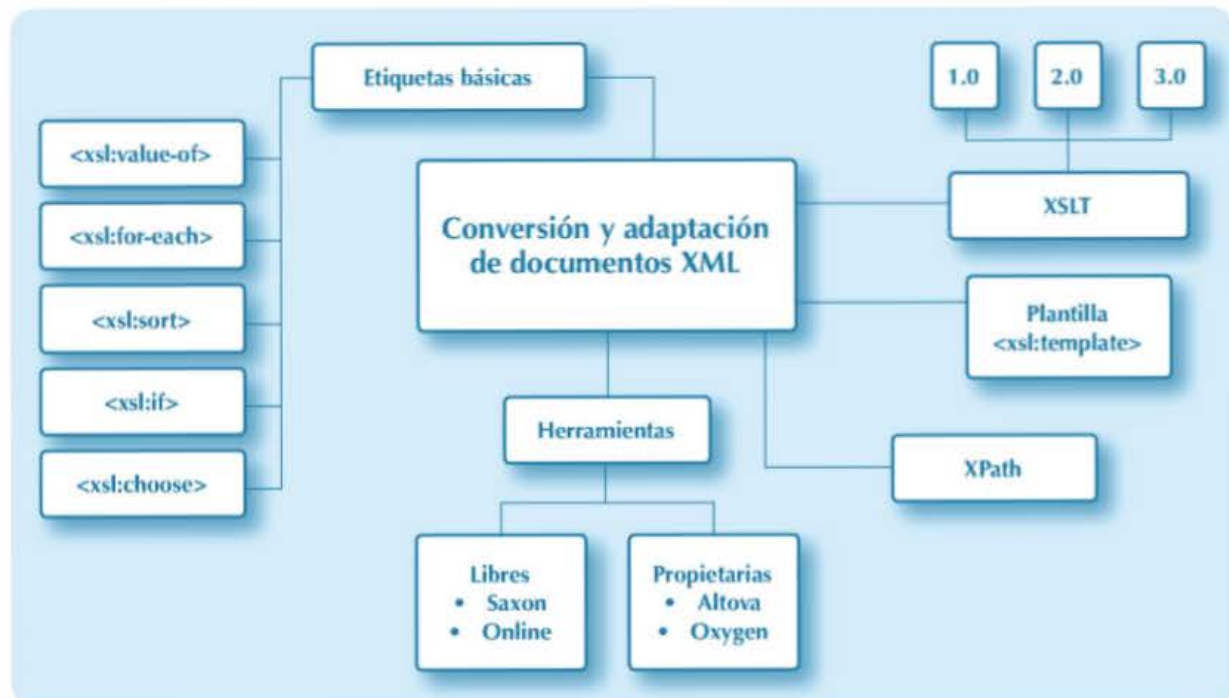


Conversión y adaptación de documentos XML

Objetivos

- ✓ El objetivo de este capítulo es que aprendas a transformar documentos XML. XML es un estándar que permite definir el formato y los datos en un mismo fichero. Con XSLT combinado con otras herramientas como XPath, podrás transformar unos ficheros XML en otros o en HTML. Este tipo de herramientas a nivel profesional son muy importantes y necesarias en ciertos proyectos o trabajos.
- ✓ Es importante que conozcas las diferencias existentes entre las versiones 1, 2 y 3 de XSLT.
- ✓ También deberás aprender la sintaxis de un archivo XSL y practicar las conversiones tanto a XML como a HTML. Dentro de dichas conversiones, deberás conocer el funcionamiento y la aplicación de las etiquetas básicas XSL como `<xsl:value-of>`, `<xsl:for-each>` o `<xsl:sort>`, entre otras.

Mapa conceptual



Glosario

Expresión regular. Patrón para encontrar una combinación de caracteres dentro de un texto o una cadena.

Refactorización. Acción de reestructurar el código para una mejor comprensión, mantenibilidad, eficiencia, etc., pero sin modificar su funcionalidad.

Template. Plantilla.

W3C. Acrónimo de World Wide Web Consortium. Organismo que se encarga de regular los estándares web.

XPath. Es un lenguaje de consulta para un documento XML. Permite seleccionar nodos dentro del mismo.

XSL. Sigla de *extensible stylesheet language* (lenguaje de hojas de estilo extensibles).

XSL-FO. Sigla de *extensible stylesheet language formatting objects*.

XSLT. Sigla de *XML stylesheets language for transformation* (lenguaje de transformación basado en hojas de estilo).

5.1. Introducción

XSL es la sigla de *extensible stylesheet language* (lenguaje de hojas de estilo extensibles). Básicamente, es un lenguaje para la transformación de documentación. Se basa en los siguientes pilares:

- **Transformación.** Convirtiendo los ficheros XML en otro formato.
- **Formateo de objetos.** A través del diseño de un documento XML.
- **Búsqueda de contenido en documentos.** A través del árbol de nodos de un documento XML.

El XSL es parte del XML y basa su versatilidad en tres componentes fundamentales:

1. **XPath**. Es un lenguaje para seleccionar o navegar entre los nodos en lenguaje XML dentro de un DOM.
2. **XSL-FO**. FO de formato de objetos (*formatting objects*). Transformaciones para convertir el documento XML en otro tipo de información como HTML o PDF (el famoso formato de Adobe).
3. **XSLT**. Sigla de *XML stylesheets language for transformation* (lenguaje de transformación basado en hojas de estilo).

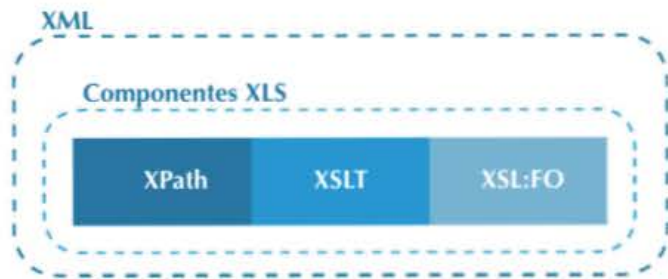


Figura 5.1
Componentes de XML.



SABÍAS QUE...

Actualmente, existen tres versiones de XSLT: la 1.0, 2.0 y la 3.0.

XSLT se utiliza para las transformaciones y es la parte más importante de XSL, por lo que se considera una recomendación del W3C (World Wide Web Consortium).

El objetivo de XSLT es convertir los documentos XML en:

- Otros documentos XML.
- Otros formatos reconocibles por un navegador como HTML o XHTML, por ejemplo.

Generalmente, la transformación que hace XSLT es de uno a uno. Cada elemento XML se transformará en otro HTML o XHTML.

RECUERDA

- ✓ XSL es más que un lenguaje de hojas de estilo. CSS hace un documento más legible, pero que sigue siendo el mismo, mientras que XSL transforma un documento en otro.



PARA SABER MÁS

XSL-FO

Es la sigla de *extensible stylesheet language formatting objects*. Básicamente, es un lenguaje para formatear un documento XML, pero, como es muy complejo, no cuenta con mucho soporte.

El objetivo de este lenguaje es parecido al CSS, definir la forma en la que deberá mostrarse un documento XML por pantalla (o papel).

Actividad propuesta 5.1



Marca cuáles de estos lenguajes no permiten transformar un documento:

☐ V ☐ F a) HTML.

☐ V ☐ F b) XSL.

☐ V ☐ F c) XPath.

☐ V ☐ F d) XHTML.

Verificar



5.2. Técnicas de transformación de documentos XML

Una de las ventajas de las transformaciones es que pueden añadirse o quitarse elementos y atributos del fichero de salida. Como puede deducirse, XSLT ofrece mucho juego, puesto que los elementos de un fichero pueden reestructurarse, ordenarse, etc.

Además, pueden tomarse decisiones como qué elementos van a mostrarse u ocultarse, entre otras.

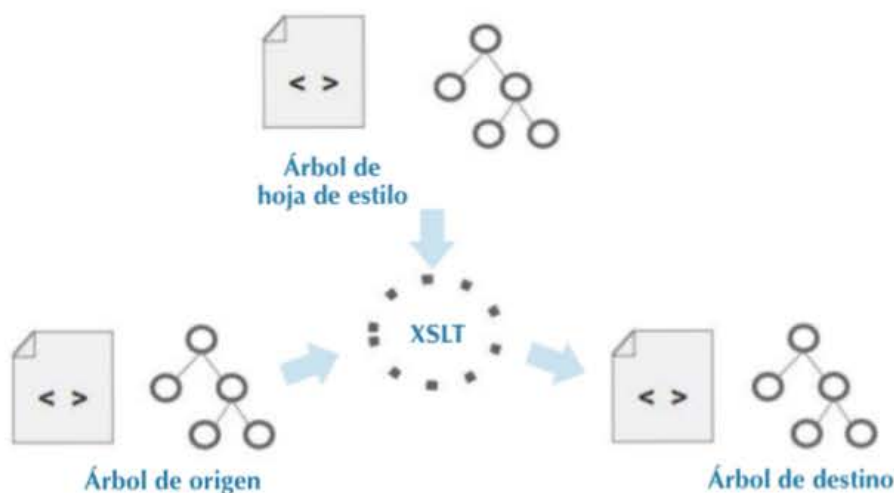


Figura 5.2

Esquema de una transformación XSLT.

Básicamente, una transformación XSLT consiste en modificar un árbol de entrada en uno de salida.

¿Qué navegadores soportan XSLT?



Una transformación XSLT se realiza usando un procesador XSLT, *software* que toma como entrada dos archivos (XML y XSL) y genera como documento de salida otro aplicando en la transformación la hoja de estilos XSL al fichero origen XML.

Los procesadores XSLT pueden estar integrados en un editor XML o en un explorador web o bien pueden invocarse desde la línea de comandos (como Xalan o Saxon).

Muchas veces, los procesadores XML son librerías que pueden estar integradas en cualquier tipo de programa.

5.2.1. Diferencias entre las versiones de XSLT 1.0, 2.0 y 3.0

XSLT 1.0 fue creada el 16 de noviembre de 1999 y, ocho años después (en el 2007), nació XSLT 2.0. Por lo tanto, aunque son retrocompatibles, la versión 2.0 tiene bastantes mejoras con respecto a la versión 1.0. Saxonica (Saxon) recomendó encarecidamente utilizar este estándar, que es compatible hacia delante (un programa que funcionaba con la versión 1.0 debería funcionar con procesadores con versiones superiores).

Actualmente, existen buenos procesadores XSLT como son Xalan, Saxon y MSXML.

Entre las novedades de la versión 2.0, están las siguientes:

- *Mejora del sistema de tipos.* La versión 1.0 era muy restrictiva, puesto que solamente existían cinco: *string*, *number*, *boolean*, *node-set* y *result tree fragment*.
- *Mejora de las funciones incorporadas y los operadores.*
- *Mejora del modelo de datos.*

Recurso web

www

Te recomendamos acceder a la web donde está publicada la recomendación del W3C:



- *Soporte para expresiones regulares.*
- *Múltiples árboles como resultado.* En la versión 1.0, solamente podía crearse un árbol de salida.
- *La instrucción <xsl:function>.* Sirve para poder definir funciones que puedan ser utilizadas en expresiones XPath.
- *Mejora del sistema de errores.*
- *Mejora de la funcionalidad de agrupación.* Por ejemplo, se añade la función <xsl:for-each-group>.

Recurso web

www

El W3C tiene toda la documentación sobre XSLT 3.0, a la que podrás acceder a través de este enlace.



Una de las mejoras que se han incorporado es la posibilidad de *streaming* de los documentos fuente. Muchas veces, los documentos son muy grandes y se necesita producir resultados antes de que todo el documento esté disponible. Para ello, se crearon nuevas instrucciones para poder utilizar esta nueva funcionalidad como <xsl:source-document>, <xsl:iterate>, <xsl:merge> o <xsl:fork>.

Asimismo se añadió modularidad a los programas creando el concepto de *paquete* como en Java. Un paquete define un interfaz que regula funciones, variables, plantillas y otros compo-

nentes que, pese a estar visibles fuera del paquete, pueden ser sobrescritas. El objetivo último es la reutilización y mantenibilidad del código.

Otras instrucciones nuevas son:

- *xsl:evaluate*. Para evaluar expresiones XPath que se construyen dinámicamente con *strings* (o que se leen de un documento).
- *xsl:try*. Para recuperarse de errores en tiempo de ejecución.
- *xsl:global-context-item*. Se utiliza cuando se necesita un elemento cuyo contexto sea global (también se declara el tipo del elemento).
- *xsl:assert*. Para ayudar a los desarrolladores a crear código robusto.

5.3. Formatos de salida y ámbitos de aplicación

Como se ha mencionado anteriormente, XSLT puede transformar un documento XML en otro XML o en otro formato, como puede ser XHTML.

A continuación, se mostrará un ejercicio guiado para ver el formato de salida de una transformación XSLT, con los siguientes pasos

1. Crear un fichero XML.
2. Crear un fichero XSL.
3. Abrir el fichero XML con un navegador.

Ejercicio práctico guiado 5.1

5.4. Descripción de la estructura y la sintaxis

Lo primero que va a hacerse en este apartado es explicar en profundidad el ejercicio práctico guiado 5.1.

5.4.1. Fichero XML

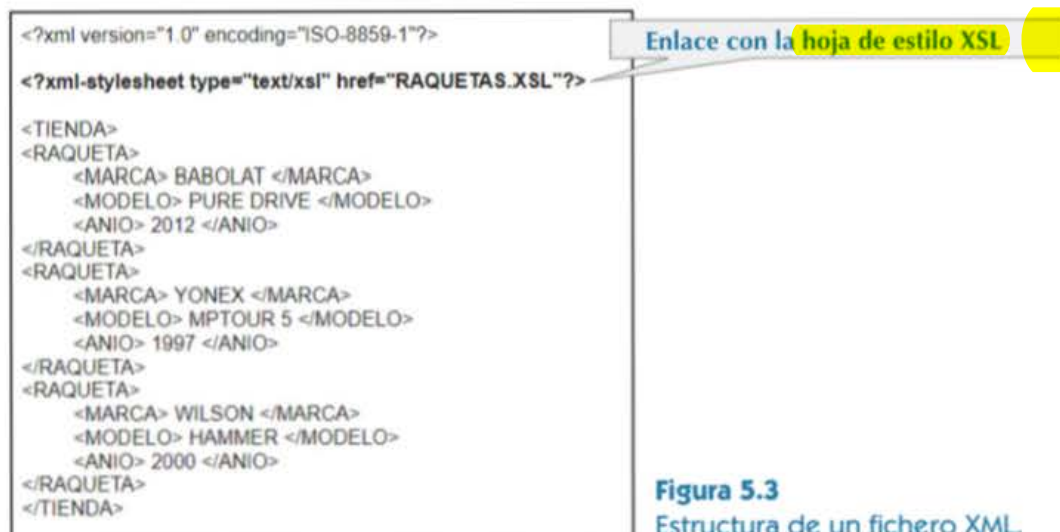


Figura 5.3
Estructura de un fichero XML.

La estructura de este fichero es similar a la del documento XML visto anteriormente, pero la diferencia radica en la **utilización de una plantilla XSL**, como puede apreciarse mediante la siguiente línea:

```
<?xml-stylesheet type="text/xsl" href="RAQUETAS.XSL"?>
```

Como puede observarse, la etiqueta indica que es una plantilla XSL y el atributo *href* con el valor **"RAQUETAS.XSL"** indica la ubicación de la propia plantilla RAQUETAS.XSL. En este caso, se encuentra en la misma ubicación que el fichero XML.

5.4.2. Hoja de estilo XSL

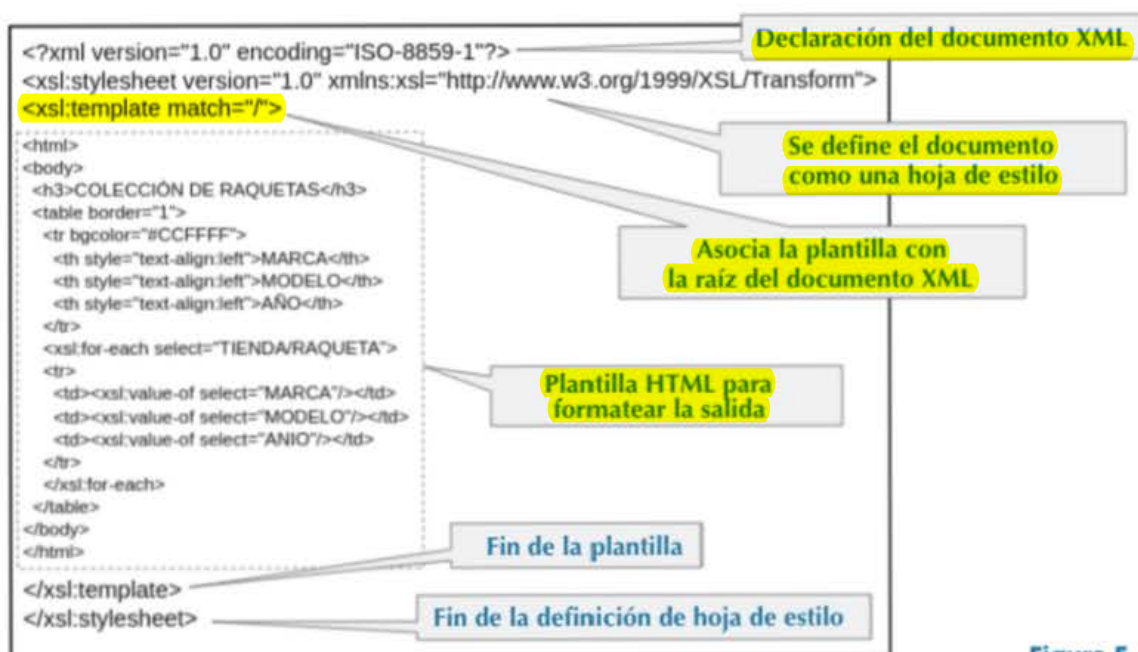


Figura 5.4
Estructura de un fichero XSL.

RECUERDA

- ✓ Una hoja de estilo XSL es una serie de reglas que se llaman **plantillas**.
- ✓ Una plantilla contendrá las acciones o reglas necesarias, que se aplicarán cuando coincida dicha regla con un nodo concreto.

En la orden siguiente del fichero XSL, pueden distinguirse dos partes:

```
<xsl:template match="/">
```

- a) La propia **orden <xsl:template>**.
 - Se utiliza para **crear plantillas**

b) El atributo *match* (*match*="/").

- Asocia una plantilla a un elemento XML.
- Normalmente, suele utilizarse para definir una plantilla para todo el documento XML. Para ello, se utiliza el atributo *match* con la expresión XPath "/", la cual indica que va a utilizarse todo el documento. En vez de la raíz, puede especificarse otra ruta en XPath.

El contenido dentro de la etiqueta `<xsl:template>` tendrá el HTML necesario para formar la salida según la necesidad del programador.

Como se ha mencionado con anterioridad, el atributo *match* utiliza una expresión XPath, que se estudiará en profundidad en el capítulo siguiente, pero, en el cuadro 5.1, se detallan algunas expresiones comunes de este lenguaje.

CUADRO 5.1

Expresiones comunes de XPath

Expresión	Resultado
/	Todo el documento.
/TIENDA	El elemento <i>TIENDA</i> del documento.
//RAQUETAS	Todos los elementos <i>RAQUETAS</i> del documento.
/TIENDA*/MARCA	Los elementos <i>MARCA</i> que son nietos del elemento <i>TIENDA</i> del documento.

Generalmente, se utiliza el primero ejemplo visto ("/") para seleccionar todo el documento, pero pueden elegirse nodos concretos. Las posibilidades de XPath son bastante amplias.

Actividad propuesta 5.2



¿Qué permite el lenguaje XPath?

- ☐ V ☐ F a) Modificar el formato de los datos de un fichero XML.
- ☐ V ☐ F b) Transformar los datos de un fichero XML.
- ☐ V ☐ F c) Acceder a los datos de un fichero XML.
- ☐ V ☐ F d) Crear una jerarquía de datos dentro de un fichero XML.

Verificar



5.5. Utilización de plantillas

La operativa básica para transformar archivos XML no es muy complicada, pero la creación de la plantilla correcta para realizar una transformación concreta puede ser más o menos difi-

cultosa, para lo cual, XSLT tiene una serie de **etiquetas**. A continuación, se describirán las más utilizadas.

5.5.1. **<xsl:value-of>**

Dentro de la plantilla XSL del ejercicio práctico guiado 5.1, pueden observarse órdenes del tipo:

```
<td><xsl:value-of select="MARCA"/></td>
<td><xsl:value-of select="MODELO"/></td>
<td><xsl:value-of select="ANIO"/></td>
```

La etiqueta `<td>` ya se ha tratado, dado que es una etiqueta HTML, pero aparece una nueva etiqueta: `<xsl:value-of>`.

Esta etiqueta extrae el valor de un elemento XML y lo añade al flujo de salida de la transformación. En el ejemplo trabajado, a la salida HTML.

Con el atributo `select`, el cual utiliza una expresión XPath, se seleccionará el elemento concreto.

5.5.2. **<xsl:for-each>**

Otra etiqueta cuyo significado es intuitivo, pero que se estudiará a continuación, es la siguiente:

```
<xsl:for-each select="TIENDA/RAQUETA">
```

En esta etiqueta `<xsl:for-each>`, se seleccionarán todos los elementos uno a uno de una lista determinada. Los elementos que van a utilizarse estarán filtrados por el atributo `select`, el cual utilizará una expresión XPath.

Ejercicio resuelto 5.1



Imagina que, en el ejercicio práctico guiado 5.1, quieren mostrarse solamente las raquetas de la marca BABOLAT. Reescribe la orden `<xsl:for-each>` del archivo XSL.



Solución 

RECUERDA

- ✓ La etiqueta `</xsl:for-each>` siempre tendrá su etiqueta de cierre `</xsl:for-each>`, por lo que no hay que olvidarse de escribirla en la plantilla XSL.



Ejercicio propuesto 5.1

Se ha modificado el fichero del ejercicio resuelto 5.1 con varios modelos más de raqueta:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="RAQUETAS.XSL"?>
<TIENDA>
  <RAQUETA>
    <MARCA> BABOLAT </MARCA>
    <MODELO> PURE DRIVE </MODELO>
    <ANIO> 2012 </ANIO>
    <PRECIO>170</PRECIO>
  </RAQUETA>
  <RAQUETA>
    <MARCA> BABOLAT </MARCA>
    <MODELO> PURE STORM</MODELO>
    <ANIO> 2013 </ANIO>
    <PRECIO>190</PRECIO>
  </RAQUETA>
  <RAQUETA>
    <MARCA> YONEX </MARCA>
    <MODELO> MPTOUR 5 </MODELO>
    <ANIO> 1997 </ANIO>
    <PRECIO>150</PRECIO>
  </RAQUETA>
  <RAQUETA>
    <MARCA> WILSON </MARCA>
    <MODELO> HAMMER </MODELO>
    <ANIO> 2000 </ANIO>
    <PRECIO>95</PRECIO>
  </RAQUETA>
  <RAQUETA>
    <MARCA> WILSON </MARCA>
    <MODELO> PRO STAFF </MODELO>
    <ANIO> 1997 </ANIO>
    <PRECIO>215</PRECIO>
  </RAQUETA>
</TIENDA>
```

Con los siguientes predicados XPath:



```
/TIENDA/RAQUETAS[1]
/TIENDA/RAQUETAS[last()]
/TIENDA/RAQUETAS[last()-1]
/TIENDA/RAQUETAS[position()<3]
/TIENDA/RAQUETAS[PRECIO>100]
/TIENDA/RAQUETAS[PRECIO>100]/MARCA
```

¿Cuál será el resultado de aplicarlos con una plantilla XSL? Comprueba el resultado creando una plantilla XSL al respecto.

Selecciona aquellas raquetas con un precio superior a 150 euros.

RECUERDA

El operador OR (||) en XPath permite seleccionar dos conjuntos de nodos. Por ejemplo, si desea seleccionarse la marca y el modelo de las raquetas de la tienda, podría utilizarse la expresión:

```
//MARCA || //MODELO
```

5.5.3. Filtrando contenido

Cuando un fichero es muy grande, lo más normal es filtrar el contenido y no trabajar con todo el árbol de nodos. Por ejemplo, trabajar solamente con las raquetas de la marca BABOLAT.

La etiqueta `<xsl:for-each>` podría modificarse de la siguiente forma para que filtre solamente las raquetas de dicha marca:

```
<xsl:for-each select="TIENDA/RAQUETA[MARCA = 'BABOLAT']">
  <tr>
    <td><xsl:value-of select="MARCA"/></td>
    <td><xsl:value-of select="MODELO"/></td>
    <td><xsl:value-of select="ANIO"/></td>
    <td><xsl:value-of select="PRECIO"/></td>
  </tr>
</xsl:for-each>
```

5.5.4. Ordenando el contenido

En este momento, se necesita que, además de la marca BABOLAT, las raquetas queden ordenadas por el precio.

Es preciso utilizar la etiqueta `<xsl:sort>` añadiendo la propiedad `select="PRECIO"`. El resultado sería añadir la siguiente línea al fichero:

```
<xsl:sort select="PRECIO"/>
```

Véase cómo quedaría el filtro anterior incluyendo la ordenación:

```
<xsl:for-each select="TIENDA/RAQUETA[MARCA = 'BABOLAT']">
  <xsl:sort select="PRECIO"/>
  <tr>
    <td><xsl:value-of select="MARCA"/></td>
    <td><xsl:value-of select="MODELO"/></td>
    <td><xsl:value-of select="ANIO"/></td>
    <td><xsl:value-of select="PRECIO"/></td>
  </tr>
</xsl:for-each>
```

RECUERDA

Se sobreentiende que los campos que se ordenan tendrán que ir en la salida. No tiene mucho sentido ordenar por un campo que, posteriormente, no va a aparecer en la salida.

5.5.5. Selección avanzada IF

Si quieren mostrarse todas las raquetas con un precio superior a 100 euros. Para esta operación, puede utilizarse la etiqueta `<xsl:if>` de la siguiente forma:

```
<xsl:if test="PRECIO>100">
```

El lugar donde hay que colocar esta sentencia es siempre dentro de la etiqueta `<xsl:for-each>`.

Véase el resultado completo:

```
<xsl:for-each select="TIENDA/RAQUETA[MARCA = 'BABOLAT']">
  <xsl:if test="PRECIO>100">
    <tr>
      <td><xsl:value-of select="MARCA"/></td>
      <td><xsl:value-of select="MODELO"/></td>
      <td><xsl:value-of select="AÑO"/></td>
      <td><xsl:value-of select="PRECIO"/></td>
    </tr>
  </xsl:if>
</xsl:for-each>
```

RECUERDA

El contenido de test siempre será una expresión XPath.

5.5.6. Selección avanzada CHOOSE

Imagínese que quiere conseguirse un resultado como el de la figura 5.5

Figura 5.5

Resultado del ejercicio que quiere conseguirse.

COLECCIÓN DE RAQUETAS

MARCA	MODELO	AÑO	PRECIO
BABOLAT	PURE DRIVE	2012	170
BABOLAT	PURE STORM	2013	190
YONEX	MPTOUR 5	1997	150
WILSON	HAMMER	2000	95
WILSON	PRO STAFF	1997	215

Dependiendo del valor de la raqueta, el precio se sombreadá de amarillo si supera el precio de 150 euros y de verde en caso contrario.

La plantilla que habrá que utilizar para realizar esta transformación es la siguiente:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Trans-
  form">
<xsl:template match="/">
<html>
<body>
  <h3>COLECCION DE RAQUETAS</h3>
  <table border="1">
    <tr bgcolor="#CCFFFF">
      <th style="text-align:left">MARCA</th>
      <th style="text-align:left">MODELO</th>
      <th style="text-align:left">AÑO</th>
      <th style="text-align:left">PRECIO</th>
    </tr>
    <xsl:for-each select="TIENDA/RAQUETA">
      <tr>
        <td><xsl:value-of select="MARCA"/></td>
        <td><xsl:value-of select="MODELO"/></td>
        <td><xsl:value-of select="AÑO"/></td>
        <xsl:choose>
          <xsl:when test="PRECIO>150">
            <td bgcolor='#fff66'><xsl:value-of select="PRE-
              CIO"/></td>
          </xsl:when>
          <xsl:otherwise>
            <td bgcolor='#99ff99'><xsl:value-of select="PRE-
              CIO"/></td>
          </xsl:otherwise>
        </xsl:choose>
      </tr>
    </xsl:for-each>
  </table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

Se ha utilizado la etiqueta `<xsl:choose>` de tal manera que, al seleccionar las raquetas con precio mayor a 150 euros (`<xsl:when test="PRECIO>150">`), la salida tendrá un sombreado y, en caso contrario (`<xsl:otherwise>`), el sombreado será diferente.

5.6. Utilización de herramientas de procesamiento: verificación, depuración y generación de documentación

Existen muchas herramientas para la edición de documentos XML. Las mejores herramientas son de pago, mientras que las menos potentes suelen ser *open source*.

www

Recurso web

A través de este enlace podrás acceder a Free Online XSLT Tool, que es una herramienta *online* para hacer transformaciones XSLT.



¿Cómo usar Free Online XSLT Tool?

A continuación, se hablará de las dos herramientas de *software* comercial más utilizadas en la actualidad.

Actividad propuesta 5.3

Responde si la siguiente afirmación es verdadera o falsa:

V **F**

Para transformar un documento mediante XSLT, es necesario utilizar un editor XML, ya sea propietario u *open source*, con un procesador XSLT incorporado.

Verificar 



PARA SABER MÁS

Los depuradores

Los depuradores son herramientas que permiten seguir paso a paso la ejecución de la transformación de un archivo XML. Permiten localizar y corregir errores de una manera más rápida y cómoda que si no se dispusiese de depurador.

Solo están disponibles en herramientas potentes que, normalmente, son de pago como Oxygen, Altova o EditiX.

Con estas herramientas, es posible ver la plantilla que está ejecutándose, los elementos XML implicados en la transformación y qué salida está produciendo dicha transformación.

5.6.1. Oxygen XML Editor

El editor XSLT de Oxygen es uno de los más utilizados por los desarrolladores XSLT. Pueden realizarse todas las acciones necesarias con los documentos XSLT, como puede ser su creación, edición, testeo o validación.

Funciones del editor XSLT de Oxygen XML Editor

5.6.2. Altova Editor XSLT

El editor XSLT de Altova tiene todos los aspectos de un editor avanzado como el coloreado de sintaxis, plegamiento de código, marcadores, finalización de código, etc.

Asimismo tiene funciones de herramientas avanzadas como la generación y evaluación de XPath, transformación de alta velocidad y explorador web integrado.

Funciones de Altova Editor XSLT

Recursos web

www

A través de los siguientes enlaces podrás acceder a las páginas de Oxygen XML Editor, Altova Editor XSLT y XPontus XML Editor.



PARA SABER MÁS

Aceleradores de la velocidad de transformación

En muchas herramientas de nivel profesional, el *kernel* del sistema de transformación tiene un acelerador u optimizador de velocidad. Eso quiere decir que, cuando los ficheros son muy grandes, hay ciertos cuellos de botella que hacen que las transformaciones sean lentas.

Estos optimizadores de velocidad realizan cambios en los programas (refactorización) sin que se vea afectada la funcionalidad ni el formato de salida con unos patrones predefinidos. De tal manera que, si el optimizador constata que dichos cambios mejoran la velocidad de transformación, los realiza y, si ve que no suponen ninguna mejora, los obvia.

5.6.3. Saxon

Saxon es un procesador XSLT desarrollado por Saxonica Limited. Tiene varias versiones (*home edition*, *professional edition* y *enterprise edition*). De estas versiones, solamente la *home-edition* es *open source* bajo licencia Mozilla Public License versión 2.0. La versión más completa es la *enterprise edition*.

Funciones de Saxon

www

Recursos web

A través de estos enlaces podrás acceder a la página web de Saxon, desde la que puedes descargar cualquier versión de esta herramienta, y a Saxonica, donde podrás encontrar más información sobre cómo utilizar esta herramienta.



Ejercicio resuelto 5.2

Transforma el fichero RAQUETAS.XML del ejercicio práctico guiado 5.1 con la herramienta Saxon.

Solución

5.6.4. XPontus XML Editor

XPontus es un editor XML *open source* no muy sofisticado, pero que tiene muchas funciones que pueden ayudar a un programador a realizar un pequeño proyecto con XML.

Funciones de XPontus**Ejercicio práctico guiado 5.2. XPontus**

5.6.5. Herramientas online

Se recomienda utilizar **XSL Test Online**, que puede servir para hacer transformaciones en línea desde cualquier dispositivo.

Estas herramientas tienen la ventaja de poder ejecutarse desde prácticamente cualquier entorno, pero no ofrecen las mismas funcionalidades que una herramienta de escritorio como Altova u Oxygen, por ejemplo.

Para una programación intensiva o a nivel profesional, este tipo de herramientas a veces se quedan cortas. Sobre todo a la hora de depurar errores. Es más difícil depurar cuando la herramienta ayuda poco.

Ejercicio práctico guiado 5.3. XSLT 2.0 con FreeFormatter XSL

Recurso web

www

Con este enlace podrás acceder a la página web de **FreeFormatter XSL**, herramienta que, al igual que **XSL Test Online**, es muy intuitiva y prácticamente ofrece un servicio de procesado XSLT sin ninguna funcionalidad añadida.



Resumen

- Los ficheros XML permiten organizar la información de forma jerárquica. Estos documentos son legibles, pero hay que procesar la información para darle un valor añadido o hacerla comprensible para cualquier persona.
- Se necesita alguna herramienta para transformar esa información XML en otro documento XML o en un documento que pueda leerse o imprimirse. Ahí entra en juego XSLT, que es un estándar para transformar esos documentos XML.
- XSLT va por la versión 3.0, lo que implica la importancia de este estándar.
- Con XSL, puede transformarse, formatearse y buscarse contenido en ficheros XML.
- XPath se utiliza para navegar entre nodos (seleccionar nodos).
- XSL-FO permite formatear objetos XML.
- XSLT permite transformar un fichero XML en otro documento como HTML o XML.
- XSL es más que un lenguaje de hojas de estilo. CSS hace un documento más legible, pero sigue siendo el mismo), mientras que XSL transforma un documento en otro.
- Una de las ventajas de la versión 3.0 de XSLT es el *streaming* de los documentos fuente.
- Etiqueta del fichero XML que hace referencia a la hoja de estilo XSL:

```
<?xml-stylesheet type="text/xsl" href="RAQUETAS.XSL"?>
```

- Orden template:

```
<xsl:template match="/">
```

- Se distingue la orden y el atributo *match* que asocia en XPath la plantilla a un nodo XML.
- Etiquetas más usuales:
 - `<xsl:value-of>`. Extrae el valor de un elemento XML y lo añade al flujo de salida de la transformación.
 - `<xsl:for-each>`. Selecciona todos los elementos uno a uno de una lista determinada.
 - `<xsl:sort>`. Ordena alfabéticamente un grupo de nodos.

- `<xsl:if>`. Estructura condicional. Se ejecutará su contenido siempre y cuando se cumpla la condición del IF.
- `<xsl:choose>`. Estructura condicional parecida a la anterior, pero permite optar entre múltiples condiciones.
- Herramientas de procesamiento:
 - *De pago*. Tienen depurador, se utilizan a nivel profesional y poseen muchas funciones.
 - Oxygen.
 - Altova.
 - Editix.
 - *Open source*. Más modestas, tienen buenos procesadores, pero un IDE no tan logrado.
 - Saxon.
 - Xalan.
 - Xpontos.
 - *Online*. Disponibles en cualquier dispositivo y sistema, aunque tienen pocas funciones.
 - Xsl Test Online.
 - FreeFormatter XSL.

Ejercicios prácticos resueltos

1. Teniendo el siguiente fichero XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<ciclos>
  <ciclo codigo="ASIR">
    <nombre>Administración de Sistemas Informáticos en Red</nombre>
    <grado>Superior</grado>
  </ciclo>
  <ciclo codigo="DAM">
    <nombre>Desarrollo de Aplicaciones Multiplataforma</nombre>
    <grado>Superior</grado>
  </ciclo>
  <ciclo codigo="DAW">
    <nombre>Desarrollo de Aplicaciones Web</nombre>
    <grado>Superior</grado>
  </ciclo>
  <ciclo codigo="SMR">
    <nombre>Sistemas Microinformáticos y Redes</nombre>
    <grado>Medio</grado>
  </ciclo>
</ciclos>
```

Realiza un fichero XSL que produzca la siguiente salida:

```
<html>
  <body>
    <p>
      <strong>Administración de Sistemas Informáticos en Red</strong>
    </p>
    <p>
      <strong>Desarrollo de Aplicaciones Multiplataforma</strong>
    </p>
    <p><strong>Desarrollo de Aplicaciones Web</strong></p>
    <p><strong>Sistemas Microinformáticos y Redes</strong></p>
  </body>
</html>
```

Solución 

2. Modifica el fichero XSL para que ahora la salida sea la siguiente:

```
<html>
<body>
  <ul>
    <li>
      <strong>
        Administración de Sistemas Informáticos en Red
      </strong>
    </li>
    <li>
      <strong>Desarrollo de Aplicaciones Multiplataforma</strong>
    </li>
    <li>
      <strong>Desarrollo de Aplicaciones Web</strong>
    </li>
    <li>
      <strong>Sistemas Microinformáticos y Redes</strong>
    </li>
  </ul>
</body>
</html>
```

Solución 

3. Cambia el fichero XSL para que ahora la salida sea la siguiente:

```

<html>
  <body>
    <ul>
      <li>
        <strong>
          ASIR:Administración de Sistemas Informáticos en Red
        </strong>
      </li>
      <li>
        <strong>
          DAM:Desarrollo de Aplicaciones Multiplataforma
        </strong>
      </li>
      <li>
        <strong>DAW:Desarrollo de Aplicaciones Web</strong>
      </li>
      <li>
        <strong>SMR:Sistemas Microinformáticos y Redes</strong>
      </li>
    </ul>
  </body>
</html>

```

Solución

4. Transforma el fichero XSL para que ahora la salida sea la siguiente:

```

<html>
  <body>
    <h1>GRADO SUPERIOR</h1>
    <ul>
      <li>
        <strong>
          ASIR:Administración de Sistemas Informáticos en Red
        </strong>
      </li>
      <li>
        <strong>
          DAM:Desarrollo de Aplicaciones Multiplataforma
        </strong>
      </li>
      <li>
        <strong>DAW:Desarrollo de Aplicaciones Web</strong>
      </li>
    </ul>
  </body>
</html>

```

Solución

Ejercicios prácticos



1. Teniendo el siguiente fichero XML:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl"?>
<hello-world>
  <yo>Juan Carlos Moreno</yo>
  <saludo>¡Hola Mundo!</saludo>
</hello-world>
```

Y el siguiente fichero XSL:

```
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
  <xsl:template match="/hello-world">
    <HTML>
      <HEAD>
        <TITLE></TITLE>
      </HEAD>
      <BODY>
        <H1> <xsl:value-of select="saludo"/> </H1>
        <xsl:apply-templates select="yo"/>
      </BODY>
    </HTML>
  </xsl:template>
  <xsl:template match="yo">
    <DIV>
      De: <strong><xsl:value-of select="."/> </strong>
    </DIV>
  </xsl:template>
</xsl:stylesheet>
```

Crea la salida mediante más de alguna herramienta citada en el capítulo (se recomienda utilizar XPontus y alguna herramienta *online* como FreeFormatter XSL).

2. Disponiendo del siguiente fichero XML:

```
<?xml version="1.0" encoding="utf-8"?>
<?xml-stylesheet type="text/xsl" href="discos.XSL"?>
<CATALOGO>
  <DISCO>
    <TITULO>ME VA LA VIDA</TITULO>
    <ARTISTA>JULIO IGLESIAS</ARTISTA>
    <PAIS>ESPANA</PAIS>
    <COMPANIA>Kamaleon</COMPANIA>
    <PRECIO>9.80</PRECIO>
    <ANIO>1985</ANIO>
  </DISCO>
```

[.../...]

```

<DISCO>
  <TITULO>GRANDES EXITOS</TITULO>
  <ARTISTA>UN PINGUINO EN MI ASCENSOR</ARTISTA>
  <PAIS>PORTUGAL</PAIS>
  <COMPANIA>Pingu-records</COMPANIA>
  <PRECIO>14.80</PRECIO>
  <ANIO>1992</ANIO>
</DISCO>
<DISCO>
  <TITULO>UNCHAIN MY HEART</TITULO>
  <ARTISTA>JOE COCKER</ARTISTA>
  <PAIS>USA</PAIS>
  <COMPANIA>EMI</COMPANIA>
  <PRECIO>9.20</PRECIO>
  <ANIO>1987</ANIO>
</DISCO>
</CATALOGO>

```

Desea conseguirse el siguiente formato HTML de salida:

```

<html>
  <body>
    <h1>CATALOGO DE DISCOS</h1>
    <table>
      <tr>
        <th>TITULO</th>
        <th>ARTISTA</th>
      </tr>
      <tr>
        <td>ME VA LA VIDA</td>
        <td>JULIO IGLESIAS</td>
      </tr>
      <tr>
        <td>GRANDES EXITOS</td>
        <td>UN PINGUINO EN MI ASCENSOR</td>
      </tr>
      <tr>
        <td>UNCHAIN MY HEART</td>
        <td>JOE COCKER</td>
      </tr>
    </table>
  </body>
</html>

```

Crea el fichero XSL de transformación.

3. Modifica el ejercicio práctico 2 para que solamente aparezcan los discos de Portugal.
4. Transforma el ejercicio práctico 2 para conocer el dinero que vale la colección completa.
5. Añade varios discos al catálogo y modifica el ejercicio práctico 2 para que aparezcan solamente los discos del año 1990 en adelante.
6. Muestra todos los discos en una tabla, pero el fondo será verde si el precio es mayor a 10 euros y amarillo en caso contrario.

